Multi-Source Candidate Data Transformer - Design Document

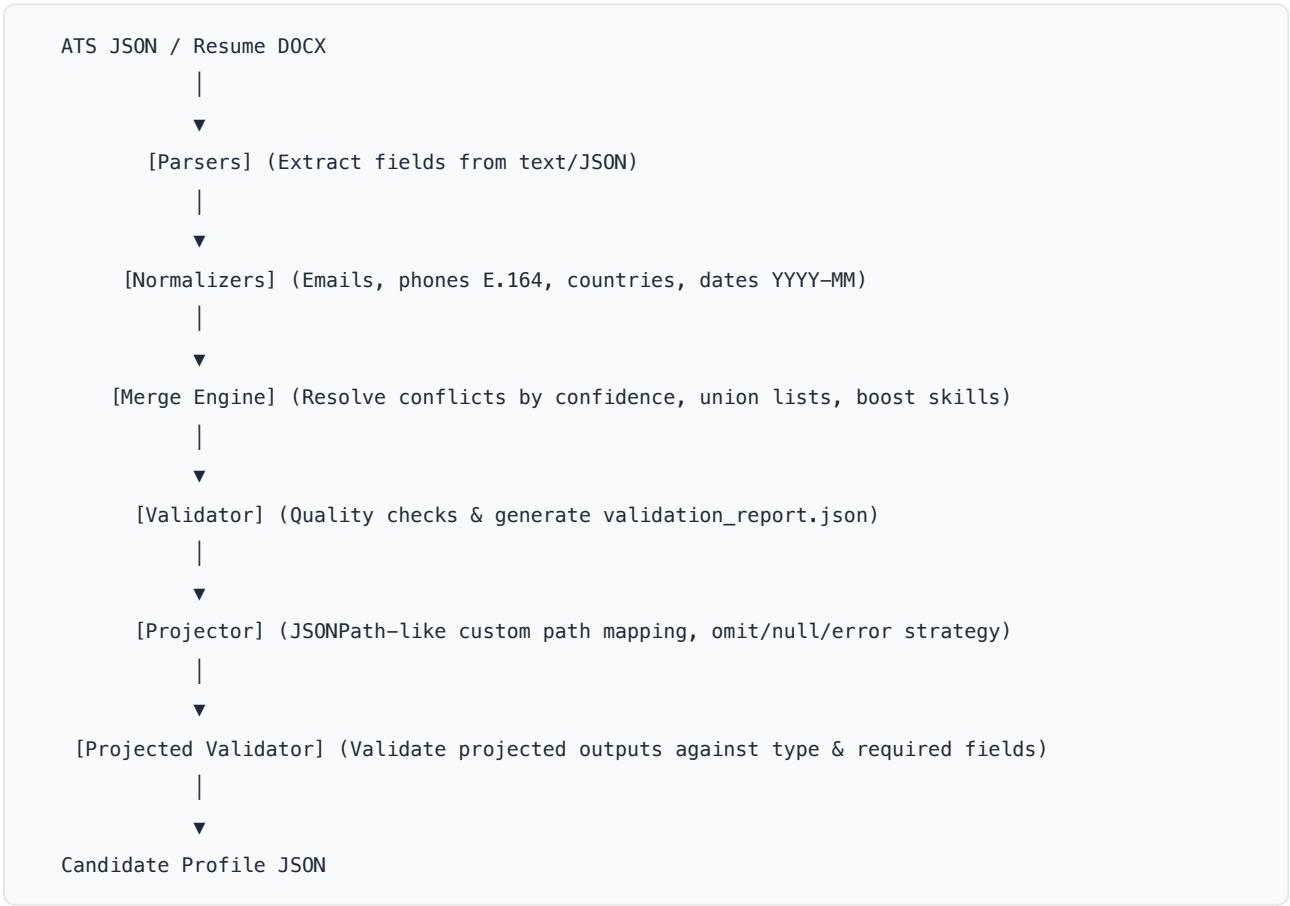
Author: Pillikandla Kruthin Reddy
Assignment: Eightfold Engineering Intern Assignment (Jul–Dec 2026)

Problem Statement

Ingesting candidate information from multiple heterogeneous sources (like ATS exports, resume files, social profiles) usually introduces data duplication, inconsistencies, conflicts, and formatting anomalies. The objective of this project is to build a robust, modular, and configurable candidate data transformation pipeline that resolves these conflicts, normalizes data formats, tracks provenance, and projects custom output schemas based on runtime configuration.

Pipeline Architecture

The pipeline processes data through 7 distinct lifecycle stages:



Key Components

1. Ingestion & Parsers (`src/parsers/`)

- **ATS JSON Parser**: Maps semi-structured fields to the canonical model using alias groups. Fallbacks exist to synthesize experiences from root fields (`current_company` , `title`) when a history block is absent.
- **Resume Parser**: Extracts raw text from `docx` , `pdf` , `txt` , `json` , and `md` .
- **Entity Extractor**: Scans text using pattern matching for emails, phones (utilizing `phonenumbers`), and links.
- **Section Segmenter**: Splits text based on common section headers.
- **Experience & Education Heuristics**: Parses companies, role titles, and education records into structured lists.
- **Years of Experience Calculator**: Calculates the absolute calendar span of all experience intervals in unique months (converting year/month dates and tracking unique month coverage to prevent inflation due to concurrent internships).

2. Normalization Layer (`src/normalizers/`)

- **Emails**: Stripped and converted to lower case.
- **Phones**: Formatted to E.164 using the `phonenumbers` library with country fallback (e.g. `+91`).
- **Country**: Mapped to ISO-3166 alpha-2 standard (e.g. `India` -> `IN`).
- **Dates**: Normalized from text descriptions (e.g., `May 2025` or `05/2025`) to canonical `YYYY-MM` or `Present` .
- **Skills**: Cleaned of bullets/brackets and standardized using a canonical `SKILL_MAP` (e.g. `py` -> `Python`).

3. Merge Engine (`src/merger.py`)

- **Conflict Resolution**: Resolves single-value fields (name, headline, location, years_experience) by picking the value from the source with the highest confidence score.
- **Primary Contact Selection**: Resolves list-like fields (emails, phones) by selecting the primary contact detail from the highest confidence source, keeping it clean and recruiter-friendly.
- **Skill Confidence Boosting**: For duplicate skills, selects the max source confidence and adds an `agreement_bonus` (default `+0.10`) for each independent source that contains the skill, up to `1.0` .
- **Provenance**: Inserts `{ field, source, method }` for each resolved field.
- **Deduplication**: Automatically groups and merges duplicate educational and experience entries.

4. Custom Projector (`src/projector.py`)

- Reshapes intermediate canonical records based on configuration without code modifications.
- Supports:
- Select list index: `emails[0]` , `phones[0]`
- Nested list extraction: `skills[].name`
- Per-field runtime normalizations (e.g., `"E164"` , `"canonical"` , `"upper"` , `"lower"`)
- Missing field strategies: `"null"` (fill with null), `"omit"` (exclude from JSON), `"error"` (abort pipeline)

5. Dual Validation (`src/validator.py`)

- **Canonical Validation**: Checks if critical fields (ID, Name, Email, Phones, Skills) are present and whether the overall confidence score is above `0.5` . Saves a quality audit in `output/validation_report.json` .
- **Projected Validation**: Confirms the final projected output conforms to specified field types and `required` parameters.

Edge Case Handling

1. **Concurrent Internships**: Calculated using a calendar-month tracking set to prevent overlap inflation.
2. **Missing Source Data**: Gracefully degrades (e.g., if ATS JSON is missing, resume ingestion still succeeds).
3. **Invalid Phone/Email Formats**: Automatically falls back to raw values if standard parsing libraries fail.
4. **Missing Required Fields**: The pipeline raises strict validation errors or omits/nullifies based on runtime settings.